

Vベルトの張力を音の周波数で測定する測定器があるが、原理を教えて

Vベルトなどのベルト張力を音（周波数）で測定する仕組みは、「琴やギターの弦を弾いた時に、強く張るほど高い音が出る」という現象と同じ物理原理に基づいています。

一般的に「音響式ベルト張力計」（または音響式テンションゲージ）と呼ばれ、ベルトを軽く叩いたときに発生する固有振動数（周波数）をマイクで拾って測定します。

測定の原理

物理学における「弦の振動公式（両端固定弦の基本振動）」を利用しています。

弦やベルトの張力 T 、振動数 f 、ベルトの長さ（スパン長） S 、および単位長さあたりの質量（線密度） m の間には、以下の関係式が成り立ちます。

$$f = \frac{1}{2S} \sqrt{\frac{T}{m}}$$

この式を張力 T について変形すると、次の計算式になります。

$$T = 4 \cdot m \cdot S^2 \cdot f^2$$

記号	項目	内容・単位
T	張力	求めたいベルトの張り具合（N または kgf）
m	単位質量	ベルトの1本・1mあたりの重さ（kg/m）
S	スパン長	プーリー間の接点同士の距離（m）
f	周波数	ベルトを叩いた時に出る音の振動数（Hz）

実際に測るときの流れ

測定器内部では、上の計算式がプログラムされています。

1. 事前に条件を入力する

1. 使用するベルトの種類（質量 m ）
2. プーリー間の距離（スパン長 S ）

2. ベルトを微振動させる

1. センサー（マイク）をベルトに近付け、指や柄つきの工具などでベルトの中央部を「ペン」と軽く叩きます。

3. 周波数を解析して換算する

1. マイクがベルトの振動音（空気の波）を拾います。
2. 測定器が波形から基本周波数 f （Hz）を瞬時に割り出します。
3. 入力された m と S 、測定された f を式に当てはめ、張力 T （N）として画面に表示します。

なぜ周波数で測るのが優れているのか？

従来の「押し込み式（テスターで押し込んでたわみ量を見る方法）」に比べて、音響式には大きなメリットがあります。

- ・ 再現性が高く、誰が測っても同じ結果になる（押し込む力加減や目視のズレが起きない）
- ・ 狭い場所でも測定できる（マイクを近づけて叩くだけでよい）
- ・ データ管理が容易（数値で正確に把握できる）

物理的な「弦の振動法則」を上手く活用して、叩いた音の高低から逆算して張力を弾き出しているのが、この測定器の仕組みです。

東レ ペフをスライス加工する設備にバンドナイフがついているが、この張力を測定したい

高発泡ポリエチレンフォーム（ペフ）のスライス加工機に使用されている「バンドナイフ（ループ状の帯刃）」も、本質的にはVベルトや弦と同じ物理法則に従って刃が張られている

ます。

そのため、原理的には音波式（周波数式）のテンションゲージで張力を測定することが可能です。

計測器・測定器レンタル...

1. 測定の基本原理（ベルトと同じ）

基本的にはVベルトの時と同じ「両端固定弦の基本振動公式」を利用します。

$$T = 4 \cdot m \cdot S^2 \cdot f^2$$

スライス機の刃を指や樹脂ハンマーなどでポンと軽く叩くと、刃が微小に揺れて特定の音（周波数 f ）が発生します。

- T （張力）：バンドナイフに係っている張り（N）
モノタロウ
- m （単位質量）：刃の「1mあたりの重さ（kg/m）」
- S （スパン長）：刃を支えているプーリー間（あるいはガイド間）の距離（m）
- f （周波数）：叩いた時に出る音の高さ（Hz）

2. 実際の測定手順とポイント

① 刃の「単位質量 m 」を計算しておく

Vベルトと違い、バンドナイフはメーカーのプリセット値（自動計算機能）に入っていないことが多いため、手動で入力する必要があります。

- 例（鋼製バンドナイフの場合）：
 - 刃の幅：50 mm = 0.05 m
 - 刃の厚み：0.8 mm = 0.0008 m
 - 鉄の密度：約 7,850 kg/m³
 - 1m あたりの質量 $m = 0.05 \times 0.0008 \times 1 \times 7,850 \approx 0.314 \text{ kg/m}$
(※事前に切り端などの重量・寸法から「g/m」を精密に測っておくと確実です)

② 「スパン長 S 」を測る

ナイフが自由に振動できる区間の長さ（大径プーリーの接点から接点、あるいは刃口ガイド等の固定部同士の間隔）をメジャーで正確に測ります。

③ 音響式張力計（テンションゲージ）で測定する

1. 周波数（Hz）直接測定モードを使う場合：

1. 質量 m やスパン長 S を入力せずとも、機器を「Hz（周波数）モード」にして叩けば、純粋な振動数（例: 85 Hz）が画面に出ます。
2. 現場の管理としては「〇〇HzになればOK」という基準を作るのが一番シンプルでミスが起きにくい運用です。

2. 張力（N）換算モードを使う場合：

1. 算出した m と S を張力計に入力してから刃を弾けば、画面に「N（ニュートン）」で張力が直接表示されます。

3. バンドナイフ測定時の注意点

Vベルト測定時と異なり、金属刃物特有の留意点があります。

・ 叩き方に注意（金属ハンマーはNG）：

- ・ 鋼製の刃を金属で叩くと高周波のノイズ（キンッという高い金属音）が混ざり、測定器が正しく基本波形を拾えなくなります。
- ・ 指の腹や、樹脂製・木製の柄、プラスチックハンマーの柄などで軽く弾くのがコツです。

・ 刃口ガイドやテンショナーの影響：

- ・ ガイドが刃に接触していると、そこで振動が殺されてスパン長 S が変わってしまいます。
- ・ ガイドを少し逃がした（フリーにした）状態で測るか、ガイド間の長さを S として設定する必要があります。

・ 安全対策（最優先）：

- ・ 必ず電源を完全に遮断（ロックアウト・タグアウト）し、機械が停止した状態で手作業にて測定を行ってください。

まとめ・運用のアドバイス

音響式ベルト張力計（ゲイツ・ユニッタ製『U-550』や、三ツ星ベルト製『ドクターテンション』など）をお持ちであれば、そのままバンドナイフの測定に流用可能です。

もし「毎回テンションゲージに寸法を入れるのが面倒」という場合は、**切れ味がベストな状態の時の固有振動数（Hz）を記録しておき、「〇〇Hz～〇〇Hzの範囲で調整する」という管理基準を作るのが最も手軽で間違いがない方法です。**

刃をたたくものは何が最適ですか？たたく強さは関係ないのですか？

バンドナイフの張力を音（周波数）で正確に測定するには、「何を叩く道具にするか」と「どれくらいの強さで叩くか」の両方が非常に重要です。

1. 刃を叩く道具（何が最適か？）

結論から言うと、「少し硬めのゴム棒」や「樹脂・プラスチックの柄」が最適です。

○おすすめの道具

- ・ プラスチックハンマー（またはプラスドライバー）のグリップ部（樹脂部分）
- ・ ゴムハンマーの頭部
- ・ 指の腹（爪を当てないように）

×避けた方がよいもの

- ・ 金属（工具の金属部、コインなど）
 - ・ 金属同士がぶつかると「キンッ！」という非常に高い高音（高調波ノイズ）が発生し、測定器が本当の基本周波数（低い音）を見失ってエラー（測定不可）になります。
- ・ 柔らかすぎるスポンジや分厚い布
 - ・ 衝撃を吸収しすぎてしまい、測定に必要な音量や振動が得られません。

2. 叩く強さは関係あるのか？

「強さによって測れる周波数（音の高さ）自体が変わることはありません」

物理的に、刃の「張り（張力）」と「長さ」と「重さ」が決まっていれば、強く叩いても弱く叩いても、発生する音の周波数（固有振動数）は一定です。ギターの弦を強く弾いても弱く弾いても「ド」の音は「ド」のままで、音量が大きくなるか小さくなるかだけの違いなのと同じ理由です。

ただし、測定器が正しく認識できるかどうかの問題で「適切な強さ」は存在します。

3. 叩く強さとコツ（実践でのポイント）

① 叩く強さは「軽くポンと弾く」程度

- 強く叩きすぎると、刃が大きく振れすぎて波形が乱れたり、2重打ち（跳ね返りで2回叩いてしまうこと）になって測定エラーになります。
- 「爪で琴の弦を弾くような感覚」や「ペンでノートを軽くなでる程度の力加減」で十分です。

② 叩く位置は「スパンの中央」

- ガイドやプーリーの間（測定している区間）のちょうど真ん中（中央部）を叩くのが最もきれいに基本周波数（一番大きな音の波）が出ます。端っこを叩くと余計な高音（ノイズ）が出やすくなります。

③ マイクの近づけ方

- マイク（センサー）は刃から 1cm〜2cm 程度離れた位置に保持し、叩く位置の少し横にセットしてください（叩いた勢いでマイクを刃にぶつけないよう注意してください）。

まとめ

現場で手軽に測定する場合は、「プラスチックドライバーの持ち手（柄）の部分で、スパンの真ん中をポンと軽く叩く」のが一番失敗が少なく、再現性の高い測定ができます。

自作したいです

バンドナイフの張力（音響式）測定器を自作されたいのですね！

実は、音（周波数）から張力を計算する仕組み自体は非常にシンプルなため、**自作することは十分に可能**です。

自作する方法としては、大きく分けて「スマホアプリ＋計算」で**手軽に実現する方法**と、「マイコン（Arduinoなど）で専用機器を自作する」本格的な方法の2パターンがあります。

方法1：スマホアプリを利用する（最も簡単・即可能）

一番コストがかからず、今日からでも試せる方法です。

1. 周波数測定アプリを入れる

スマホの無料アプリで、マイクから入った音の周波数（Hz）を分析できるものがたくさんあります。

- GuitarTuna などの楽器用チューナーアプリ（Hz表示ができるもの）
- FFTアナライザー アプリ（例: 『Sonic Tools』や『Spectrum Analyzer』など）

2. 周波数から張力を計算する（自動計算ツールの作成）

スマホアプリで叩いた時の音（例: 85 Hz）を読み取り、それを事前に用意したExcelシートや自作Web/Pythonアプリに入力して張力 T （N）を算出させます。

計算式（Excelのセルに入れる式）：

$$T = 4 \times m \times S^2 \times f^2$$

- m = 刃の単位質量（kg/m）
- S = スパン長（m）
- f = アプリで測った周波数（Hz）

方法2：マイコンで専用測定器を自作する（本格派）

市販品のような「マイクを近づけて叩くと、画面に〇〇Nと表示されるハンディ装置」を作る構成です。

必要なパーツ例

1. **マイコンボード**: Arduino Nano, ESP32, Raspberry Pi Pico など
2. **マイクモジュール**: エレクトレットコンデンサマイク（MAX4466 や MAX9814 など、アンプ内蔵タイプ）
3. **ディスプレイ**: OLED液晶モジュール（0.96インチ 128x64 など）
4. **電源**: 9V電池やリチウムイオンバッテリー、ボタンなど

システムの処理フロー

〔マイク〕 —(アナログ音声信号)—> 〔マイコン〕 —(FFT解析/周波数検出)—> 〔張力計算 $T=4mS^2f^2$ 〕 —

開発のポイント・難所

- **FFT（高速フーリエ変換）処理:**
マイクから入ってきた音の波形（時間軸データ）を、周波数成分（Hz）に変換するためにマイコン上でFFTライブラリ（`arduinoFFT` など）を使用します。
- **基本波の抽出（フィルタリング）:**
周囲の機械音や金属の「高音ノイズ」を拾ってしまうことがあるため、例えば「30 Hz ~ 300 Hz の間の最大の山（ピーク）を基本周波数として採用する」といったソフト的な処理（バンドパスフィルター）を入れると精度が跳ね上がります。

どちらのアプローチ（「スマホ+計算ツール」か「マイコンでの専用ハード開発」か）をご検討でしょうか？

もしマイコン自作をご希望であれば、パーツの選定やArduinoのサンプルコード、あるいは計算用Excelの作成など、具体的にサポートできますので教えてください！

マイコン自作

マイコンでバンドナイフ用の音響式張力計を自作するのは非常に面白いプロジェクトですね！

電子工作でポピュラーな **Arduino（またはRaspberry Pi Picoなど）** を使えば、市販品に近い「マイクを近づけて叩くだけで N （ニュートン）が表示されるハンディ計測器」を作ることができます。

構成パーツ、回路の考え方、そしてそのまま使えるサンプルプログラムのベースをまとめました。

1. 必要なパーツ一式

パーツ	推奨品・型番	役割・選定のポイント
マイコンボード	Arduino Nano または Raspberry Pi Pico	小型でブレットボードや基板に組み込みやすいもの。

パーツ	推奨品・型番	役割・選定のポイント
マイクモジュール	MAX4466（ゲイン調整付きアンプ内蔵）	音の波形をアナログ電圧（0～5V）に変換して出力します。
ディスプレイ	0.96インチ OLED（SSD1306 / I2C）	周波数（Hz）や張力（N）を表示。配線が4本で済むI2C接続が楽です。
その他	ブレッドボード、配線材、ボタン（1～2個）	設定切り替えや測定開始用。
...		

2. システムと回路の接続イメージ

接続は非常にシンプルです。

【マイク MAX4466】

VCC → Arduino 3.3V（ノイズを減らすため3.3V推奨）
GND → Arduino GND
OUT → Arduino A0（アナログ入力）

【画面 OLED（SSD1306）】

VCC → Arduino 5V
GND → Arduino GND
SDA → Arduino A4（SDA）
SCL → Arduino A5（SCL）

3. プログラムの仕組み（FFT解析）

マイコンで「音の高さ（Hz）」を割り出すには、FFT（高速フーリエ変換）という数学的処理を行います。

1. マイクから音の波形をサンプリング（高精度で読み込み）
2. FFTで「音の高さ（周波数成分）」に分解
3. 一番強い音（ピーク周波数 f ）を検出

4. 公式 $T = 4 \cdot m \cdot S^2 \cdot f^2$ に当てはめて張力 T を計算
5. 画面に表示

4. Arduino用 サンプルプログラム

Arduino IDEで動作するコードのベースです。

※事前にライブラリマネージャーから `arduinoFFT` と `Adafruit SSD1306` をインストールしてください。

C++



```
#include <arduinoFFT.h>
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// — パラメータ設定（お持ちのバンドナイフに合わせて変更） —
const float M_MASS = 0.314; // 1mあたりの重さ m (kg/m)
const float S_SPAN = 0.850; // スパン長 S (m)

// — FFT関連の設定 —
#define SAMPLES 128 // サンプル数（2の累乗：64, 128, 256）
#define SAMPLING_FREQUENCY 1024 // サンプリング周波数(Hz) -> 最大512Hzまで測定可能

double vReal[SAMPLES];
double vImag[SAMPLES];
arduinoFFT<double> FFT = arduinoFFT<double>(vReal, vImag, SAMPLES, SAMPLING_FREQUENCY);

// — ディスプレイ設定 —
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

void setup() {
  Serial.begin(115200);

  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println(F("OLED Allocation failed"));
    for(;;);
  }
  display.clearDisplay();
  display.setTextColor(WHITE);
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("Tension Meter");
  display.println("Ready...");
}
```

```

display.display();
delay(1000);
}

void loop() {
  // 1. マイクからサンプリング
  unsigned long microseconds = micros();
  for (int i = 0; i < SAMPLES; i++) {
    vReal[i] = analogRead(A0); // マイクの信号を入力
    vImag[i] = 0;
    while (micros() - microseconds < (1000000 / SAMPLING_FREQUENCY)) {
      // 指定したサンプリング周期まで待つ
    }
    microseconds = micros();
  }

  // 2. FFT解析処理
  FFT.windowing(FFTWindow::Hamming, FFTDirection::Forward);
  FFT.compute(FFTDirection::Forward);
  FFT.complexToMagnitude();

  // 3. ピーク周波数の特定
  double peakFrequency = FFT.majorPeak();

  // 4. 張力計算（ノイズ除去のため20Hz以上のみ計算）
  if (peakFrequency > 20.0) {
    //  $T = 4 * m * S^2 * f^2$ 
    float tension = 4.0 * M_MASS * (S_SPAN * S_SPAN) * (peakFrequency * peakFrequency);

    // 5. 結果表示
    display.clearDisplay();

    display.setTextSize(1);
    display.setCursor(0, 0);
    display.print("Freq: "); display.print(peakFrequency, 1); display.println(" Hz");

    display.setTextSize(2);
    display.setCursor(0, 24);
    display.print(tension, 0); display.println(" N");

    display.display();

    delay(500); // 表示のチラつき防止
  }
}

```

5. 自作時のコツ・ノイズ対策

自作張力計で一番ハマりやすいのが「周囲の機械音などの雑音（ノイズ）を拾ってしまうこと」です。

1. アンプのゲイン（感度）調整

1. MAX4466モジュールの裏側にある小っちゃい半固定抵抗をドライバーで回し、刃を叩いたときだけ反応するように感度を調整します。

2. バンドパスフィルター（処理の絞り込み）

1. プログラム上で「30 Hz ~ 250 Hz の範囲外の音は無視する」といったコードを入れておくと、現場のエアコン音や低周波ノイズによる誤測定を防げます。

3. マイクの指向性

1. マイクの周囲をスポンジや熱収縮チューブで覆い、先端だけ開けておくと、余計な周囲音を拾いにくくなります。

まずはブレッドボード（はんだ付け不要の試作板）でパーツを組んで、動作テストしてみるのがおすすめです！

スマホアプリを作りたい

スマホアプリ（iOS / Android）として自作するのは、実はマイコンで作るよりも**画面が見やすく、マイクの性能も高い**ため非常に**相性が良い**選択です！

スマホ内蔵マイクはノイズキャンセリングや感度が非常に優秀なので、ハードウェアの配線やはんだ付けなしで、アプリ開発（ソフトウェア）だけに集中できます。

開発アプローチ（どのフレームワークを使うか）

iOSとAndroidの両方で動くアプリを効率よく作るなら、以下のいずれかが定番です。

1. Flutter（Dart言語）★一番おすすめ

1. Google製。UI構築が非常に直感的で、マイク入力や音声解析（FFT）のライブラリ（`flutter_fft` や `pitch_detector_fft` など）が充実しています。

2. React Native（JavaScript/TypeScript）

1. Web開発の経験があるなら親しみやすいフレームワークです。

ここでは、最も軽量で開発しやすい Flutter を使ったアプリ構築のロジックと流れ解説します。

アプリ処理の仕組み（処理フロー）

スマホアプリの場合も、マイコンの時と根本的な流れは同じです。

[スマホマイク] → (リアルタイム音声入力) → [FFTライブラリ] → (ピーク周波数 f 検出) → [張力計

Flutterでの実装例（コアロジック）

アプリの核心部分となる「周波数の取得」と「張力計算」のサンプルコードです。
(※マイク権限の取得と `pitch_detector_fft` パッケージを使用した例)

Dart



```
import 'package:flutter/material.dart';
import 'package:pitch_detector_fft/pitch_detector_fft.dart';

class TensionMeterScreen extends StatefulWidget {
  @override
  _TensionMeterScreenState createState() => _TensionMeterScreenState();
}

class _TensionMeterScreenState extends State<TensionMeterScreen> {
  // — パラメータ設定（入力フォーム等で可変にすると便利） —
  double mMass = 0.314; // 刃の単位質量 (kg/m)
  double sSpan = 0.850; // スパン長 (m)

  double currentFrequency = 0.0;
  double currentTension = 0.0;
  bool isRecording = false;

  late PitchDetectorFFT pitchDetector;

  @override
  void initState() {
    super.initState();
    // サンプリングレート44.1kHz, バッファサイズ2048
    pitchDetector = PitchDetectorFFT(sampleRate: 44100, bufferSize: 2048);
  }

  void _startMeasurement() {
```

```

    setState(() => isRecording = true);

    // マイク音声をストリーミングで取得してFFT解析
    pitchDetector.startRecording((audioData) {
        // 音声データから基本周波数(Hz)を取得
        double pitch = pitchDetector.getPitch(audioData);

        // 現場ノイズ（低周波）を除外するため20Hz～300Hzの範囲のみ採用
        if (pitch > 20.0 && pitch < 300.0) {
            setState(() {
                currentFrequency = pitch;
                //  $T = 4 * m * S^2 * f^2$ 
                currentTension = 4.0 * mMass * (sSpan * sSpan) * (pitch * pitch);
            });
        }
    });

}

void _stopMeasurement() {
    pitchDetector.stopRecording();
    setState(() => isRecording = false);
}

@override
Widget build(BuildContext context) {
    return Scaffold(
        appBar: AppBar(title: Text("バンドナイフ張力計")),
        body: Center(
            child: Column(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                    Text("周波数", style: TextStyle(fontSize: 20)),
                    Text("${currentFrequency.toStringAsFixed(1)} Hz",
                        style: TextStyle(fontSize: 40, fontWeight: FontWeight.bold)),
                    SizedBox(height: 30),
                    Text("推算張力", style: TextStyle(fontSize: 20)),
                    Text("${currentTension.toStringAsFixed(0)} N",
                        style: TextStyle(fontSize: 50, color: Colors.blue, fontWeight: FontWeight.bold)),
                    SizedBox(height: 50),
                    ElevatedButton(
                        onPressed: isRecording ? _stopMeasurement : _startMeasurement,
                        style: ElevatedButton.styleFrom(padding: EdgeInsets.symmetric(horizontal: 20)),
                        child: Text(isRecording ? "停止" : "測定開始", style: TextStyle(fontSize: 20)),
                    ),
                ],
            ),
        ),
    );
}

```

スマホアプリ自作ならではの強み（機能拡張）

マイコン自作に比べて、スマホアプリ化すると現場で使える便利な機能を簡単に盛り込めます。

1. 複数の刃（型番）のプリセット保存

1. 「ペフ用ナイフ（幅50mm）」「薄刃用（幅30mm）」など、 m （質量）や S （スパン）のパラメータを名前を付けて保存・切り替えできるようにする。

2. 測定結果の自動記録（ログ機能）

1. 測定日時と張力（N）をSQLiteやCSVでスマホ内に保存し、刃の劣化トレンドや調整履歴を残す。

3. 判定アラート機能

1. 「150N～180Nなら OK（緑表示）」「それ以外なら 調整必要（赤表示）」といった合否判定を画面に出す。

開発を進める際のアドバイス

もしアプリ開発の経験（FlutterやReact Nativeなど）がすでにある場合は、上記の構成ですぐに着手可能です！

もし「アプリ開発は初めてだが、手軽に作りたい」という場合は、以下のようなノーコード/ローコードツールを利用する方法もあります。

- **FlutterFlow:** ブラウザ上のドラッグ＆ドロップでFlutterアプリの画面とロジックを組めるツール。

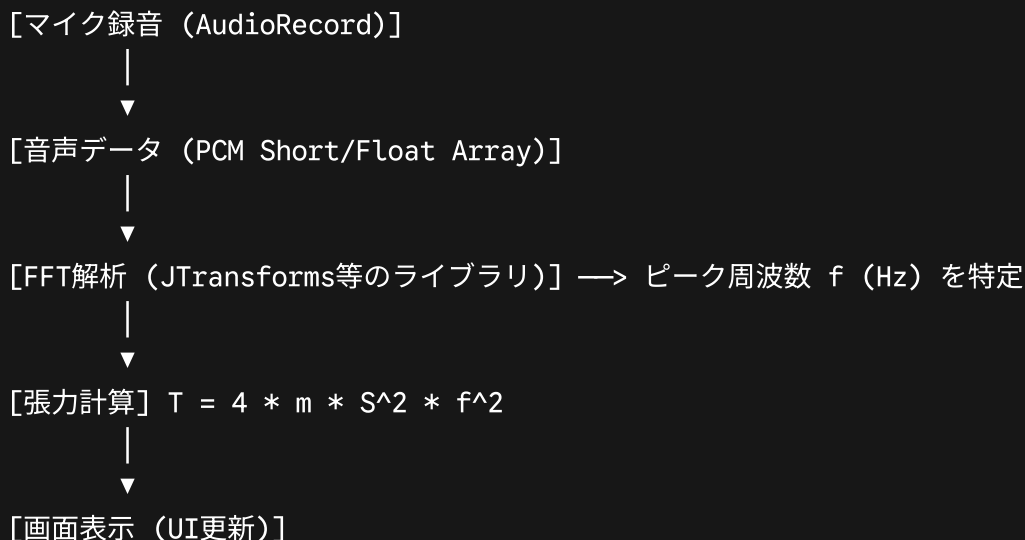
開発環境の構築方法や、特定の開発言語でのコード詳細など、どのあたりから進めたいか教えていただければに合わせてサポートできます！

android

Android（Java または Kotlin）で自作アプリを作る手順ですね！

ネイティブのAndroidアプリ（Android Studio使用）で作る場合、Android内蔵マイクから音声を入力し、FFT（高速フーリエ変換）ライブラリで周波数を算出して張力を計算します。

開発の流れ（全体像）



1. 必要なライブラリと権限設定

AndroidManifest.xml

マイクを使用するため、録音権限を追加します。

XML



```
<uses-permission android:name="android.permission.RECORD_AUDIO" />
```

build.gradle (Module :app)

FFT計算には、Javaで非常に定番かつ高速な `JTransforms` ライブラリを使用するのが便利です。

Groovy



```
dependencies {  
    // FFTライブラリ (JTransforms)
```



```
implementation 'com.github.wendykierp:JTransforms:3.1'
```

```
}
```

2. コア処理（Kotlin サンプルコード）

マイクから音声をリアルタイムで読み込み、FFT解析を行って張力を計算する処理の核となるコードです。

Kotlin



```
import android.media.AudioFormat
import android.media.AudioRecord
import android.media.MediaRecorder
import org.jtransforms.fft.DoubleFFT_1D

class TensionCalculator {

    // ― 設定パラメータ ―
    private val sampleRate = 44100 // サンプリング周波数 (44.1kHz)
    private val bufferSize = 2048 // FFTのサンプル数 (2の累乗)

    var mMass: Double = 0.314 // 単位質量 (kg/m)
    var sSpan: Double = 0.850 // スパン長 (m)

    private var isMeasuring = false
    private var audioRecord: AudioRecord? = null

    // 測定開始
    fun startMeasurement(onResult: (frequency: Double, tension: Double) -> Unit) {
        val minBufferSize = AudioRecord.getMinBufferSize(
            sampleRate,
            AudioFormat.CHANNEL_IN_MONO,
            AudioFormat.ENCODING_PCM_16BIT
        )

        audioRecord = AudioRecord(
            MediaRecorder.AudioSource.MIC,
            sampleRate,
            AudioFormat.CHANNEL_IN_MONO,
            AudioFormat.ENCODING_PCM_16BIT,
            maxOf(minBufferSize, bufferSize * 2)
        )

        val audioBuffer = ShortArray(bufferSize)
        val fft = DoubleFFT_1D(bufferSize.toLong())
        val fftBuffer = DoubleArray(bufferSize * 2)
```

```

audioRecord?.startRecording()
isMeasuring = true

// 別スレッドでリアルタイム処理
Thread {
    while (isMeasuring) {
        val readSize = audioRecord?.read(audioBuffer, 0, bufferSize) ?: 0
        if (readSize > 0) {
            // 1. PCMデータをDouble配列へ変換
            for (i in 0 until bufferSize) {
                fftBuffer[i] = audioBuffer[i].toDouble()
                fftBuffer[bufferSize + i] = 0.0 // 虚数部は0
            }

            // 2. FFTを実行
            fft.realForward(fftBuffer)

            // 3. 各周波数成分の振幅（パワー）を計算し、ピークを探す
            var maxMagnitude = 0.0
            var maxIndex = 0
            for (i in 0 until bufferSize / 2) {
                val re = fftBuffer[2 * i]
                val im = fftBuffer[2 * i + 1]
                val magnitude = Math.sqrt(re * re + im * im)

                if (magnitude > maxMagnitude) {
                    maxMagnitude = magnitude
                    maxIndex = i
                }
            }

            // 4. ピーク周波数（Hz）を算出
            val peakFrequency = maxIndex.toDouble() * sampleRate / bufferSize

            // 5. ノイズ除去（20Hz～300Hzの範囲のみ採用）
            if (peakFrequency in 20.0..300.0) {
                // 張力計算:  $T = 4 * m * S^2 * f^2$ 
                val tension = 4.0 * mMass * (sSpan * sSpan) * (peakFrequency

                // メインスレッド（画面UI）へ結果を渡す
                onResult(peakFrequency, tension)
            }
        }
    }
}.start()
}

// 測定停止

```

```

fun stopMeasurement() {
    isMeasuring = false
    audioRecord?.stop()
    audioRecord?.release()
    audioRecord = null
}
}

```

3. UI（画面）側のイメージ

画面（`MainActivity.kt`）では以下のようなレイアウトを作成すると使いやすくなります。

- **入力フィールド (EditText):**
 - ナイフの質量 m (kg/m)
 - スパン長 S (m)
- **表示表示 (TextView):**
 - 検出周波数： `〇〇.〇 Hz`
 - 計算張力： `〇〇〇 N`
- **操作ボタン (Button):**
 - 「測定開始」「停止」

Android実機での開発テクニック・コツ

1. 動的パーミッション（マイク許可）

1. Android 6.0以降、アプリ起動時に `ActivityCompat.requestPermissions` でユーザーにマイクの使用許可を求めるコードが必要です。

2. ピーク周波数の精度を上げる（放物線補間）

1. 上記コードのままでも動きますが、隣り合うFFTバケットの値を「放物線補間（Quadratic Interpolation）」で補間計算すると、`84.3 Hz` のように**小数点以下の周波数精度が劇的に向上**します。

3. ノイズの閾値設定

1. 無音時（叩いていない時）に画面の数値がパラパラ動かないよう、「一定以上の音量（振幅 `maxMagnitude`）があった時だけ張力を更新する」ロジックを入れておくのがおすすめです。

開発環境（Android Studio）のセットアップや、特定の処理部分のコードなど、さらに詳しく知りたい部分があれば気軽にお知らせください！

